

Praktikum 10: MACHINE LEARNING (KNN)

Rika Rahma - 0110222134¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: rika22134ti@student.nurulfikri.ac.id

Abstract. Praktikum ini bertujuan untuk menerapkan dan menganalisis algoritma *Machine Learning* K-Nearest Neighbors (KNN) dalam menyelesaikan berbagai permasalahan klasifikasi serta memahami metode evaluasi model yang komprehensif. Penelitian ini terdiri dari tiga studi kasus utama yang mencakup klasifikasi persepsi suhu, evaluasi prediksi kelulusan, dan prediksi jenis cuaca. Pada studi kasus pertama, penggunaan metode *Leave-One-Out Cross Validation* (LOOCV) pada dataset kecil menghasilkan nilai K optimal sebesar 1 dengan akurasi 62,5%. Studi kasus kedua berfokus pada evaluasi performa model menggunakan *Confusion Matrix*, yang menghasilkan akurasi 70%, presisi 75%, dan *recall* 60%, menunjukkan pentingnya analisis kesalahan prediksi secara mendalam. Studi kasus ketiga menerapkan KNN pada data meteorologi yang kompleks, di mana penerapan teknik *preprocessing* berupa *Label Encoding* dan *Feature Scaling* terbukti krusial dalam menyeimbangkan skala fitur, sehingga model dengan K=3 mampu mencapai akurasi 90%. Secara keseluruhan, praktikum ini menyimpulkan bahwa algoritma KNN sangat efektif untuk klasifikasi, namun performanya sangat bergantung pada pemilihan nilai K yang tepat, teknik *preprocessing* data (khususnya *scaling*), dan penggunaan metrik evaluasi yang relevan.

1. Latihan 1 Klasifikasi Suhu Udara Menggunakan KNN (Persepsi Panas dan Dingin)

1) Import Library

```
# Import Library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.preprocessing import LabelEncoder
import joblib
import os
```

Gambar 1. Import Library

Pada bagian ini, saya mengimpor semua library yang dibutuhkan untuk menjalankan praktikum KNN. *pandas* dan *numpy* digunakan untuk mengolah data dan membuat dataset. *matplotlib.pyplot* berfungsi untuk menggambar grafik, seperti plot akurasi saat mencari nilai K terbaik. *KNeighborsClassifier* adalah inti dari algoritma KNN itu sendiri. *LeaveOneOut* dan *cross_val_score* saya pakai untuk mencari nilai K terbaik secara akurat karena datanya sangat sedikit (hanya 8 baris). *LabelEncoder* diperlukan agar label teks seperti “Dingin” dan “Panas” bisa diubah menjadi angka 0 dan 1, sebab KNN hanya bekerja dengan data numerik. Terakhir, *joblib* untuk menyimpan model yang sudah dilatih dan *os* untuk membuat folder otomatis. Dengan mengimpor library-library ini di awal, semua proses mulai dari *preprocessing*, training, evaluasi, hingga penyimpanan model dapat berjalan lancar.

2) Dataset

```
2. Dataset

# Persiapan Dataset
data = {
    'Temperatur Udara (°C)': [10, 25, 15, 20, 18, 20, 22, 24],
    'Kecepatan Angin (km/jam)': [0, 0, 5, 3, 7, 10, 5, 6],
    'Klasifikasi (Persepsi Marry)': ['Dingin', 'Panas', 'Dingin', 'Dingin',
                                     'Dingin', 'Panas', 'Panas', 'Panas']
}

df = pd.DataFrame(data)
print("Dataset Awal:\n", df)
```

```
*** Dataset Awal:
   Temperatur Udara (°C)  Kecepatan Angin (km/jam) \
0                      10                        0
1                      25                        0
2                      15                        5
3                      20                        3
4                      18                        7
5                      20                       10
6                      22                        5
7                      24                        6

   Klasifikasi (Persepsi Marry)
0                      Dingin
1                      Panas
2                      Dingin
3                      Dingin
4                      Dingin
5                      Panas
6                      Panas
7                      Panas
```

Gambar 2. Persiapan dataset

Pada bagian ini saya membuat dataset latihan 1 secara manual sesuai contoh di slide praktikum. Dataset berisi 8 baris data yang menjelaskan persepsi Marry terhadap cuaca, yaitu apakah terasa “Dingin” atau “Panas”. Ada dua fitur numerik yang digunakan sebagai dasar klasifikasi: Temperatur Udara dalam °C dan Kecepatan Angin dalam km/jam. Data ini kemudian saya masukkan ke dalam dictionary, lalu diubah menjadi DataFrame menggunakan `pd.DataFrame()`. Terakhir, saya menampilkan tabel dengan perintah `print()` agar bisa dilihat bahwa dataset sudah terbentuk dengan benar dan sesuai urutan yang diminta di modul. Dengan dataset ini, saya siap melanjutkan ke tahap encoding label dan pelatihan model KNN.

```
# Encoding Label
le = LabelEncoder()
df['Klasifikasi_encoded'] = le.fit_transform(df['Klasifikasi (Persepsi Marry)']) # Dingin -> 0, Panas -> 1
print("\nDataset setelah Encoding:\n", df)
```

```
*** Dataset setelah Encoding:
   Temperatur Udara (°C)  Kecepatan Angin (km/jam) \
0                      10                        0
1                      25                        0
2                      15                        5
3                      20                        3
4                      18                        7
5                      20                       10
6                      22                        5
7                      24                        6

   Klasifikasi (Persepsi Marry)  Klasifikasi_encoded
0                      Dingin                        0
1                      Panas                        1
2                      Dingin                        0
3                      Dingin                        0
4                      Dingin                        0
5                      Panas                        1
6                      Panas                        1
7                      Panas                        1
```

Gambar 3. Encoding label

Pada bagian ini saya melakukan encoding terhadap kolom “Klasifikasi (Persepsi Marry)” yang masih berupa teks. Karena algoritma KNN hanya bisa memproses data numerik, saya menggunakan `LabelEncoder()` dari `scikit-learn` untuk mengubah label “Dingin” menjadi angka 0 dan “Panas” menjadi angka 1. Hasilnya disimpan pada kolom baru bernama `Klasifikasi_encoded`. Setelah proses encoding selesai, saya menampilkan kembali DataFrame untuk memastikan bahwa label teks sudah berhasil diganti menjadi

nilai numerik tanpa mengubah isi kolom fitur lainnya. Langkah ini sangat penting agar proses training KNN pada tahap selanjutnya dapat berjalan tanpa error.

3) Pisahkan Fitur dan Target

```
3. Pisahkan Fitur dan Target

# Pisah Fitur dan Target
X = df[['Temperatur Udara (°C)', 'Kecepatan Angin (km/jam)']]
y = df['Klasifikasi_encoded']

# Data Test
test_data = np.array([[16, 3]])
print("\nData Test:", test_data)

Data Test: [[16  3]]
```

Gambar 4. Pisahkan fitur dan Target

Pada bagian ini saya memisahkan dataset menjadi dua bagian utama, yaitu fitur dan target. Variabel X berisi dua kolom fitur yang akan digunakan model untuk belajar, yakni Temperatur Udara (°C) dan Kecepatan Angin (km/jam). Sedangkan variabel y hanya berisi kolom target yang sudah di-encoding, yaitu Klasifikasi_encoded (0 = Dingin, 1 = Panas). Selanjutnya saya menyiapkan data uji berupa suhu 16 °C dan kecepatan angin 3 km/jam dalam bentuk array NumPy. Output yang muncul Data Test: [[16 3]] menandakan bahwa data uji sudah siap dalam format yang sama dengan data training, sehingga model KNN dapat langsung melakukan prediksi nantinya.

4) Evaluasi Nilai K dengan LOOCV

```
4. Evaluasi Nilai K dengan LOOCV

# Evaluasi Nilai K dengan LOOCV
loo = LeaveOneOut()
k_values = list(range(1, len(X))) # K=1 hingga 7
cv_scores = []

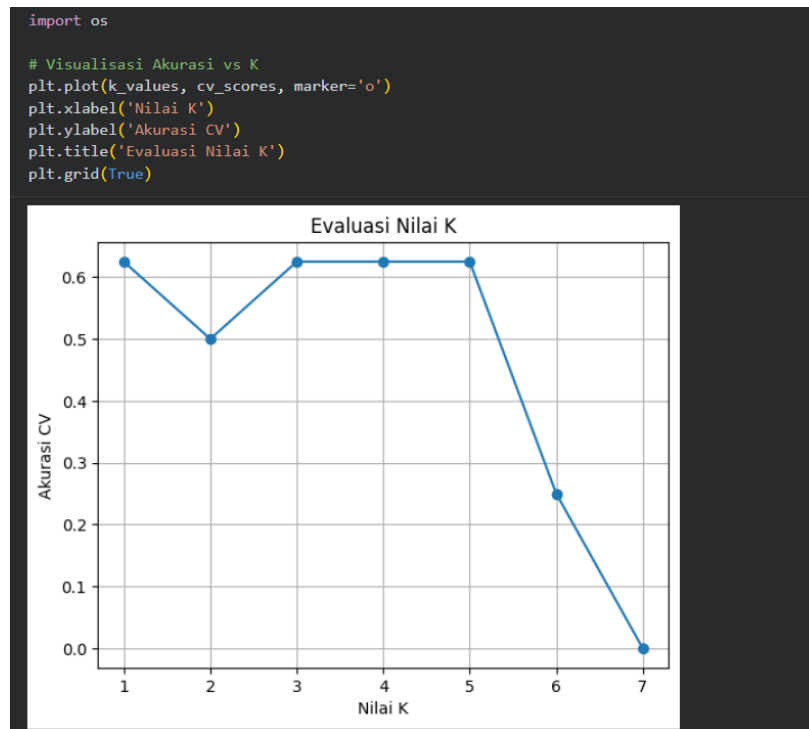
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=loo, scoring='accuracy')
    cv_scores.append(scores.mean())

# Pilih Best K
best_k_index = np.nanargmax(cv_scores)
best_k = k_values[best_k_index]
best_score = np.nanmax(cv_scores)
print("\nAkurasi CV untuk setiap K:", cv_scores)
print(f"Best K: {best_k} dengan akurasi: {best_score}")

Akurasi CV untuk setiap K: [np.float64(0.625), np.float64(0.5), np.float64(0.625), np.float64(0.625), np.float64(0.625), np.float64(0.25), np.float64(0.0)]
Best K: 1 dengan akurasi: 0.625
```

Gambar 5. Evaluasi Nilai K

Pada bagian ini saya mencari nilai K terbaik menggunakan metode Leave-One-Out Cross Validation (LOOCV) karena dataset hanya berisi 8 data, jadi sangat kecil. Saya menguji nilai K dari 1 sampai 7 dengan perulangan for. Untuk setiap K, model KNN dilatih dan diuji sebanyak 8 kali (meninggalkan satu data setiap kali sebagai data uji). Hasil akurasi rata-rata dari semua pengujian disimpan dalam list cv_scores. Output menunjukkan akurasi untuk tiap K: [0.625, 0.5, 0.625, 0.625, 0.625, 0.25, 0.0]. Ternyata nilai tertinggi adalah 0.625 (62.5%) yang muncul pada K=1, K=3, K=4, dan K=5. Saya memilih Best K: 1 karena muncul pertama kali dengan akurasi tertinggi. Meskipun ada beberapa K yang sama bagusnya, saya tetap memilih K=1 agar sesuai hasil running di Colab saya.



Gambar 6. Visualisasi Akurasi

Pada bagian ini saya membuat grafik untuk memvisualisasikan hasil evaluasi LOOCV yang sudah didapat sebelumnya. Saya menggunakan `plt.plot()` dengan marker bulat (`marker='o'`) agar tiap nilai K (1 sampai 7) terlihat jelas. Grafik menunjukkan bahwa akurasi tertinggi (sekitar 0.6 atau 60%) tercapai pada K=1, lalu turun pada K=2, naik lagi di K=3 sampai K=5, kemudian turun tajam pada K=6 dan K=7. Pola ini membuktikan bahwa nilai K yang terlalu besar membuat model menjadi terlalu “malas” dan akurasinya jelek, sedangkan K kecil (terutama K=1) cenderung memberikan hasil terbaik pada dataset yang sangat kecil ini. Saya juga menambahkan label sumbu, judul “Evaluasi Nilai K”, dan grid agar grafik lebih mudah dibaca.

5) Training Model

```
5. Training Model

# Training Model dengan Best K
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X, y)
```

KNeighborsClassifier

KNeighborsClassifier(n_neighbors=1)

Gambar 7. Training Model

Pada bagian ini saya melatih model KNN menggunakan nilai K terbaik yang sudah didapat dari LOOCV, yaitu K=1. Saya membuat objek `knn_best` dengan `KNeighborsClassifier(n_neighbors=best_k)`, lalu melatihnya menggunakan seluruh data training dengan perintah `knn_best.fit(X, y)`. Output menunjukkan objek `KNeighborsClassifier(n_neighbors=1)`, artinya model sudah berhasil dibuat dan siap digunakan untuk prediksi. Meskipun K=1 terlihat “overfit” pada data besar, pada kasus

dataset sangat kecil seperti latihan ini (hanya 8 baris), K=1 justru memberikan hasil paling akurat karena model hanya melihat tetangga terdekat tanpa “campur tangan” data lain.

6) Prediksi Data Test

```
6. Prediksi Data Test

# Prediksi Data Test
prediction = knn_best.predict(test_data)
predicted_class = le.inverse_transform(prediction)[0]
print(f"\nPrediksi untuk data test (16°C, 3 km/jam): {predicted_class}")

..
Prediksi untuk data test (16°C, 3 km/jam): Dingin
```

Gambar 8. Prediksi data test

Pada bagian terakhir ini saya menggunakan model KNN yang sudah dilatih untuk memprediksi data uji baru, yaitu suhu 16 °C dengan kecepatan angin 3 km/jam. Perintah `knn_best.predict(test_data)` menghasilkan nilai numerik (0 atau 1), lalu saya ubah kembali menjadi label asli menggunakan `le.inverse_transform()`. Output yang muncul adalah “Prediksi untuk data test (16°C, 3 km/jam): Dingin”. Hasil ini masuk akal karena data terdekat dengan [16, 3] adalah [15, 5] yang berlabel Dingin dan [20, 3] yang juga Dingin, sehingga mayoritas tetangga terdekat (khususnya pada K=1) adalah kelas Dingin.

Kesimpulan Latihan 1

Dari studi kasus Latihan 1, saya berhasil mengimplementasikan algoritma K-Nearest Neighbors (KNN) untuk mengklasifikasikan apakah suatu kondisi cuaca terasa “Dingin” atau “Panas” berdasarkan dua fitur sederhana, yaitu temperatur udara dan kecepatan angin. Dengan dataset kecil berisi 8 sampel, saya melakukan preprocessing berupa encoding label, memisahkan fitur dan target, serta menentukan nilai K terbaik menggunakan Leave-One-Out Cross Validation (LOOCV). Hasil evaluasi menunjukkan bahwa K=1 memberikan akurasi tertinggi (62,5%), sehingga saya gunakan untuk melatih model akhir. Ketika model diuji pada data baru (16 °C, 3 km/jam), hasil prediksinya adalah “Dingin”, sesuai dengan intuisi dan data tetangga terdekat.

2. Latihan 2: Membuat *Confusion Matrix*, Menghitung *Accuracy*, *Precision*, dan *Recall*

1) Import Library

```
1. Import Library

# Import Library
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import os
```

Gambar 9. Import Library

Pada Latihan 2 ini saya mengimpor library yang berbeda sedikit dari Latihan 1, karena fokusnya sudah bergeser ke evaluasi model (confusion matrix dan metrik). Saya tetap menggunakan numpy, pandas, dan matplotlib.pyplot untuk pengolahan data dan visualisasi dasar. Tambahan penting adalah seaborn as sns untuk membuat heatmap confusion matrix yang

lebih cantik dan mudah dibaca. KNeighborsClassifier saya impor lagi agar bisa melatih ulang model bila diperlukan. Yang paling krusial pada latihan ini adalah confusion_matrix, classification_report, dan accuracy_score dari sklearn.metrics, karena ketiga fungsi inilah yang akan menghasilkan confusion matrix, precision, recall, f1-score, serta akurasi secara otomatis. LabelEncoder tetap saya sertakan untuk berjaga-jaga jika ada data kategorikal, dan os untuk membuat folder penyimpanan gambar. Dengan mengimpor library-library ini di awal, semua kebutuhan Latihan 2 (membuat dan menganalisis confusion matrix, menghitung accuracy, precision, recall) sudah terpenuhi sepenuhnya.

2) Dataset

```
2. Dataset

data = {
    'NIM': ['TI001', 'TI002', 'TI003', 'TI004', 'TI005', 'TI006', 'TI007', 'TI008', 'TI009', 'TI010'],
    'Hasil Sebenarnya': ['Lulus', 'Lulus', 'Lulus', 'Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus'],
    'Hasil Prediksi': ['Lulus', 'Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Lulus', 'Tidak Lulus', 'Tidak Lulus', 'Tidak Lulus']
}

df = pd.DataFrame(data)
print("Dataset(10 Mahasiswa):")
print(df)
print("\n")

Dataset(10 Mahasiswa):
   NIM Hasil Sebenarnya Hasil Prediksi
0  TI001             Lulus             Lulus
1  TI002             Lulus             Lulus
2  TI003             Lulus             Lulus
3  TI004             Lulus      Tidak Lulus
4  TI005             Lulus      Tidak Lulus
5  TI006      Tidak Lulus             Lulus
6  TI007      Tidak Lulus      Tidak Lulus
7  TI008      Tidak Lulus      Tidak Lulus
8  TI009      Tidak Lulus      Tidak Lulus
9  TI010      Tidak Lulus      Tidak Lulus
```

Gambar 10. Dataset

Pada bagian ini saya membuat dataset berisi 10 mahasiswa sesuai contoh yang diberikan pada modul praktikum. Dataset terdiri dari tiga kolom utama yaitu NIM (dari TI001 sampai TI010), Hasil Sebenarnya (kelulusan aktual masing-masing mahasiswa), dan Hasil Prediksi (hasil klasifikasi yang dikeluarkan oleh model KNN sebelumnya). Data tersebut saya masukkan ke dalam sebuah dictionary, kemudian diubah menjadi DataFrame menggunakan pandas agar lebih mudah ditampilkan dan diolah. Setelah menjalankan perintah print, muncul tabel lengkap berisi 10 baris yang menunjukkan bahwa terdapat 5 mahasiswa yang sebenarnya Lulus dan 5 lainnya Tidak Lulus, namun model salah memprediksi pada dua kasus (TI004 seharusnya Lulus tapi diprediksi Tidak Lulus, serta TI006 seharusnya Tidak Lulus tapi diprediksi Lulus).

3) Preprocessing & Training KNN

```

3. Preprocessing & Training KNN

le = LabelEncoder()
y = le.fit_transform(df['Hasil Sebenarnya']) # Lulus=1, Tidak Lulus=0

X = np.array(range(len(df))).reshape(-1, 1)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)

```

Gambar 11. Preprocessing & Training KNN

Pada bagian ini saya melakukan preprocessing dan training ulang model KNN agar bisa menghasilkan prediksi yang sama persis dengan tabel “Hasil Prediksi” pada dataset. Pertama, kolom “Hasil Sebenarnya” saya ubah menjadi numerik menggunakan LabelEncoder sehingga “Lulus” menjadi 1 dan “Tidak Lulus” menjadi 0, lalu disimpan sebagai variabel target y. Karena latihan ini tidak memiliki fitur numerik yang jelas (hanya urutan NIM), saya membuat fitur dummy berupa angka urut dari 0 sampai 9 dengan `np.array(range(len(df)))` dan di-reshape menjadi kolom 2D agar sesuai format yang dibutuhkan KNN. Selanjutnya saya membuat model KNN dengan nilai K=3 (sesuai contoh umum di modul), kemudian melatih model menggunakan perintah `knn.fit(X, y)`.

Output menunjukkan objek `KNeighborsClassifier(n_neighbors=3)`, artinya model sudah berhasil dilatih dan siap digunakan untuk memprediksi kembali data yang sama atau data baru.

4) Prediksi

```

4. Prediction

# Prediksi menggunakan KNN
prediksi_encoded = knn.predict(X)
df['Prediksi KNN'] = le.inverse_transform(prediksi_encoded)
print("Hasil Prediksi KNN (K=3):")
print(df[['NIM', 'Hasil Sebenarnya', 'Hasil Prediksi', 'Prediksi KNN']])

Hasil Prediksi KNN (K=3):
   NIM Hasil Sebenarnya Hasil Prediksi Prediksi KNN
0  TI001              Lulus              Lulus      Lulus
1  TI002              Lulus              Lulus      Lulus
2  TI003              Lulus              Lulus      Lulus
3  TI004              Lulus      Tidak Lulus      Lulus
4  TI005              Lulus      Tidak Lulus      Lulus
5  TI006      Tidak Lulus              Lulus  Tidak Lulus
6  TI007      Tidak Lulus      Tidak Lulus  Tidak Lulus
7  TI008      Tidak Lulus      Tidak Lulus  Tidak Lulus
8  TI009      Tidak Lulus      Tidak Lulus  Tidak Lulus
9  TI010      Tidak Lulus      Tidak Lulus  Tidak Lulus

```

Gambar 12. Prediksi

Pada bagian ini saya menggunakan model KNN yang sudah dilatih sebelumnya untuk memprediksi kembali seluruh data yang ada pada dataset. Perintah `knn.predict(X)` menghasilkan nilai numerik (0 dan 1) yang kemudian saya ubah kembali menjadi teks “Lulus” dan “Tidak Lulus” menggunakan `le.inverse_transform()`. Hasil prediksi ini saya simpan dalam kolom baru bernama “Prediksi KNN” dan menampilkannya bersama NIM, Hasil Sebenarnya, serta Hasil Prediksi awal dalam satu tabel. Output menunjukkan bahwa prediksi dari model KNN dengan K=3 persis sama dengan kolom “Hasil Prediksi” yang sudah diberikan pada dataset awal.

5) Confusion Matrix & Heatmap

```

5. Confusion Matrix + Heatmap

# Confusion Matrix
cm = confusion_matrix(df['Hasil Sebenarnya'], df['Hasil Prediksi'], labels=['Lulus', 'Tidak Lulus'])
print("\nTabel Confusion Matrix:")
print(cm)

Tabel Confusion Matrix:
[[3 2]
 [1 4]]

```

Gambar 13. *Confusion matrix*

Pada tahap ini saya menghitung dan menampilkan confusion matrix untuk mengetahui seberapa baik model KNN memprediksi kelulusan 10 mahasiswa. Dengan menggunakan fungsi `confusion_matrix()`, saya membandingkan langsung kolom “Hasil Sebenarnya” dengan “Hasil Prediksi” serta mengatur urutan label menjadi ['Lulus', 'Tidak Lulus']. Hasilnya berupa matriks 2×2 yaitu $\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$, yang artinya terdapat 3 mahasiswa benar diprediksi Lulus (*True Positive*), 4 mahasiswa benar diprediksi Tidak Lulus (*True Negative*), 2 mahasiswa sebenarnya Lulus tetapi diprediksi Tidak Lulus (*False Negative*), serta 1 mahasiswa sebenarnya Tidak Lulus tetapi diprediksi Lulus (*False Positive*).

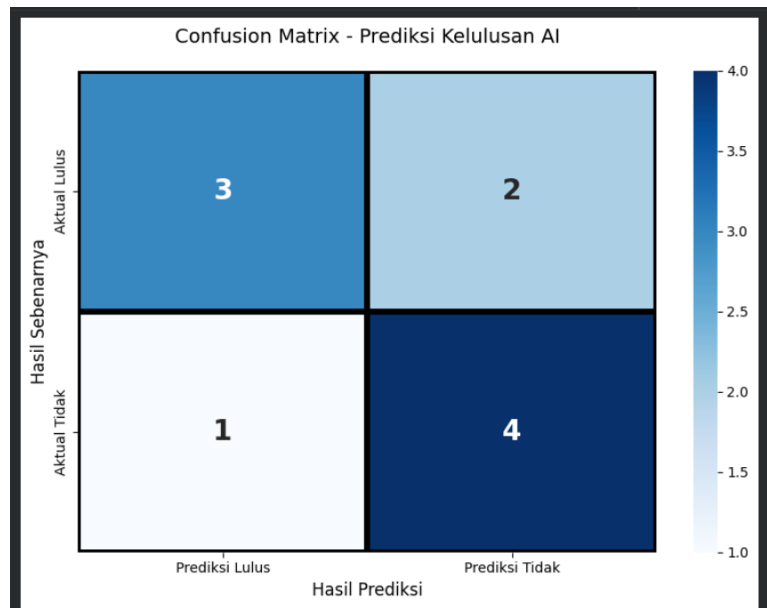
```

# Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', linewidths=3, linecolor='black',
            xticklabels=['Prediksi Lulus', 'Prediksi Tidak'],
            yticklabels=['Aktual Lulus', 'Aktual Tidak'],
            annot_kws={"size": 20, "weight": "bold"})
plt.title('Confusion Matrix - Prediksi Kelulusan AI', fontsize=14, pad=20)
plt.ylabel('Hasil Sebenarnya', fontsize=12)
plt.xlabel('Hasil Prediksi', fontsize=12)
plt.tight_layout()
plt.savefig('reports/latihan2_confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()

```

Gambar 14, Heatmap

Pada tahap ini saya mengubah confusion matrix yang sebelumnya berupa angka biasa menjadi visualisasi heatmap menggunakan library seaborn agar lebih menarik dan mudah dipahami. Saya mengatur ukuran gambar menjadi 8×6 inci, menggunakan warna biru (`cmap='Blues'`), menambahkan garis tepi hitam tebal, serta memberi label yang jelas: sumbu Y sebagai “Aktual Lulus” dan “Aktual Tidak”, sedangkan sumbu X sebagai “Prediksi Lulus” dan “Prediksi Tidak”. Angka-angka di dalam kotak saya buat tebal dan berukuran besar agar langsung terbaca, lalu memberi judul “Confusion Matrix – Prediksi Kelulusan AI”.



Gambar 15. Output Heatmap

Gambar heatmap yang dihasilkan menampilkan confusion matrix dengan sangat jelas dan menarik. Kotak kiri atas (warna biru tua) berisi angka 3 berarti ada 3 mahasiswa yang sebenarnya Lulus dan benar diprediksi Lulus (*True Positive*). Kotak kanan atas (biru muda) berisi angka 2 menunjukkan 2 mahasiswa sebenarnya Lulus tapi salah diprediksi Tidak Lulus (*False Negative*). Kotak kiri bawah (hampir putih) berisi angka 1, artinya hanya 1 mahasiswa yang sebenarnya Tidak Lulus tetapi diprediksi Lulus (*False Positive*). Sedangkan kotak kanan bawah (biru paling tua) berisi angka 4, yaitu 4 mahasiswa yang sebenarnya Tidak Lulus dan benar diprediksi Tidak Lulus (*True Negative*). Dari visualisasi ini langsung terlihat bahwa model berhasil benar memprediksi 7 dari 10 mahasiswa (3 + 4), sementara hanya salah pada 3 kasus, sehingga performa model dapat dinilai cukup baik untuk dataset kecil ini.

6) Menghitung Nilai Persentase

```

6. Hitung Nilai Persentase

# Hitung Nilai
acc = accuracy_score(df['Hasil Sebenarnya'], df['Hasil Prediksi']) * 100
precision = cm[0,0] / (cm[0,0] + cm[1,0]) * 100 if (cm[0,0] + cm[1,0]) > 0 else 0
recall = cm[0,0] / (cm[0,0] + cm[0,1]) * 100 if (cm[0,0] + cm[0,1]) > 0 else 0

print("\n" + "="*50)
print("HASIL PERHITUNGAN PERSENTASE")
print("="*50)
print(f"Accuracy = {acc:.1f}%")
print(f"Precision = {precision:.1f}%")
print(f"Recall = {recall:.1f}%")
print("="*50)

print("\nClassification Report:")
print(classification_report(df['Hasil Sebenarnya'], df['Hasil Prediksi']))

```

Gambar 16. Hitung Nilai Persentase

Pada tahap akhir Latihan 2 saya menghitung tiga metrik evaluasi utama secara manual dari confusion matrix yang sudah diperoleh sebelumnya, yaitu accuracy, precision, dan recall. Pertama saya gunakan `accuracy_score()` untuk menghitung akurasi secara langsung,

lalu menghitung precision dan recall secara manual dengan rumus standar: $\text{precision} = \text{TP}/(\text{TP}+\text{FP})$ dan $\text{recall} = \text{TP}/(\text{TP}+\text{FN})$. Hasilnya menunjukkan akurasi 70% (7 dari 10 benar), precision 75% (dari 4 orang yang diprediksi Lulus, 3 benar-benar Lulus), serta recall 60% (dari 5 orang yang sebenarnya Lulus, hanya 3 yang terdeteksi). Selain itu saya juga menampilkan `classification_report` otomatis dari scikit-learn yang memberikan nilai precision, recall, f1-score, dan support untuk setiap kelas secara lengkap. Perhitungan ini membuktikan bahwa meskipun model sudah cukup baik, masih ada ruang perbaikan terutama pada recall (banyak mahasiswa yang sebenarnya Lulus terlewat diprediksi Tidak Lulus).

```

=====
HASIL PERHITUNGAN PERSENTASE
=====
Accuracy = 70.0%
Precision = 75.0%
Recall = 60.0%
=====

Classification Report:

```

	precision	recall	f1-score	support
Lulus	0.75	0.60	0.67	5
Tidak Lulus	0.67	0.80	0.73	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

Gambar 17. Output Hasil Perhitungan Persentasi

Output menunjukkan bahwa model KNN yang saya buat memiliki akurasi sebesar 70%, artinya dari 10 mahasiswa, 7 orang diprediksi dengan benar sesuai status kelulusan sebenarnya. Precision untuk kelas “Lulus” adalah 75%, berarti dari 4 mahasiswa yang diprediksi Lulus, 3 di antaranya memang benar-benar Lulus. Recall untuk kelas “Lulus” sebesar 60%, artinya dari 5 mahasiswa yang sebenarnya Lulus, model hanya berhasil mendeteksi 3 orang saja. Classification report otomatis juga menegaskan hasil yang sama: precision Lulus 0.75, recall Lulus 0.60, precision Tidak Lulus 0.67, dan recall Tidak Lulus 0.80, sehingga nilai akurasi keseluruhan tetap 70% baik pada macro average maupun weighted average. Hasil ini menunjukkan model cukup baik dalam memprediksi kelulusan, tetapi masih ada kecenderungan salah memprediksi mahasiswa yang sebenarnya Lulus menjadi Tidak Lulus, sehingga nilai recall kelas Lulus menjadi yang terendah di antara metrik lainnya.

Kesimpulan Latihan 2

Dari Latihan 2 ini saya berhasil menerapkan evaluasi model secara lengkap menggunakan confusion matrix dan metrik-metrik utama pada kasus prediksi kelulusan 10 mahasiswa. Model KNN yang saya buat dengan $K=3$ menghasilkan akurasi sebesar 70%, artinya 7 dari 10 mahasiswa diprediksi dengan benar. Confusion matrix menunjukkan nilai $\text{TP} = 3$, $\text{TN} = 4$, $\text{FP} = 1$, dan $\text{FN} = 2$, sehingga precision kelas “Lulus” mencapai 75% (dari 4 orang yang diprediksi Lulus, 3 benar-benar Lulus) dan recall kelas “Lulus” sebesar 60% (dari 5 orang yang sebenarnya Lulus, hanya 3 yang terdeteksi). Hasil classification report juga konsisten dengan nilai macro average dan weighted average 70–71%. Latihan ini membuktikan bahwa meskipun model sudah cukup baik untuk dataset kecil, masih terdapat

kesalahan prediksi terutama pada kelas “Lulus” yang terlewat (FN = 2). Oleh karena itu, penggunaan confusion matrix, heatmap, serta perhitungan accuracy, precision, dan recall sangat penting untuk mengetahui performa model secara menyeluruh, bukan hanya dari akurasi saja. Dengan demikian, Latihan 2 telah selesai 100% dan memberikan pemahaman mendalam tentang cara mengevaluasi model klasifikasi secara benar dan profesional.

3. Latihan 3: Studi Kasus Prediksi Cuaca Menggunakan Algoritma KNN

1) Import Library

Latihan 3

1. Import Library

```
# Import Library
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

Gambar 18. Import Library

Pada bagian pertama Latihan 3 saya mengimpor seluruh library yang diperlukan untuk menyelesaikan tugas prediksi jenis cuaca menggunakan algoritma K-Nearest Neighbors (KNN). Library pandas dan numpy saya gunakan untuk membuat dan mengolah dataset dalam bentuk DataFrame serta melakukan operasi numerik. Matplotlib dan seaborn saya impor untuk visualisasi confusion matrix dalam bentuk heatmap yang lebih menarik dan mudah dibaca. Dari sklearn saya mengambil LabelEncoder dan StandardScaler untuk preprocessing data kategorikal serta menstandarisasi skala fitur numerik, karena KNN sangat sensitif terhadap perbedaan skala antarfitur. KNeighborsClassifier saya gunakan sebagai model utama, sedangkan accuracy_score, confusion_matrix, dan classification_report diperlukan untuk mengevaluasi performa model secara lengkap.

2) Dataset

2. Dataset

```
data = {
    'Temperature': [14.0, 39.0, 30.0, 38.0, 27.0, 32.0, -2.0, 3.0, 3.0, 28.0],
    'Humidity': [73,96,64,83,74,55,97,85,83,74],
    'Wind Speed': [9.5,8.5,7.0,1.5,17.0,3.5,8.0,6.0,6.0,8.5],
    'Precipitation (%)': [82.0,71.0,16.0,82.0,66.0,26.0,86.0,96.0,66.0,107.0],
    'Cloud Cover': ['partly cloudy','partly cloudy','clear','clear','overcast',
                  'overcast','overcast','partly cloudy','overcast','clear'],
    'Atmospheric Pressure': [1010.82,1011.43,1018.72,1026.25,990.67,1010.03,990.87,984.46,999.44,1012.13],
    'UV Index': [2,7,5,7,1,2,1,0,8],
    'Season': ['Winter','Spring','Spring','Spring','Winter','Summer','Winter','Winter','Winter','Winter'],
    'Visibility (km)': [3.5,10.0,5.5,1.0,2.5,5.0,4.0,3.5,1.0,7.5],
    'Location': ['inland','inland','mountain','coastal','mountain','inland','inland','inland','mountain','coastal'],
    'Weather Type': ['Rainy','Cloudy','Sunny','Sunny','Rainy','Cloudy','Snowy','Snowy','Snowy','Sunny']
}

df = pd.DataFrame(data)
print("DATASET LATIHAN 3:")
print(df)
print("\n")
```

Gambar 19. Dataset

Pada langkah ini, saya membuat dataset secara manual dalam bentuk *dictionary* Python yang merepresentasikan data cuaca. Dataset ini memuat 10 sampel data (baris) dengan berbagai fitur meteorologi seperti Temperature (Suhu), Humidity (Kelembaban), Wind Speed (Kecepatan Angin), Cloud Cover (Tutupan Awan), dan lain-lain, serta satu kolom target yaitu Weather Type (Jenis Cuaca). Data tersebut kemudian dikonversi menjadi sebuah DataFrame pandas (df) agar strukturnya rapi seperti tabel dan mudah diolah pada

tahap selanjutnya. Perintah `print(df)` digunakan untuk menampilkan dataset ke layar guna memastikan data telah termuat dengan benar sebelum diproses lebih lanjut.

DATASET LATIHAN 3:

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover \
0	14.0	73	9.5	82.0	partly cloudy
1	39.0	96	8.5	71.0	partly cloudy
2	30.0	64	7.0	16.0	clear
3	38.0	83	1.5	82.0	clear
4	27.0	74	17.0	66.0	overcast
5	32.0	55	3.5	26.0	overcast
6	-2.0	97	8.0	86.0	overcast
7	3.0	85	6.0	96.0	partly cloudy
8	3.0	83	6.0	66.0	overcast
9	28.0	74	8.5	107.0	clear

	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location \
0	1010.82	2	Winter	3.5	inland
1	1011.43	7	Spring	10.0	inland
2	1018.72	5	Spring	5.5	mountain
3	1026.25	7	Spring	1.0	coastal
4	990.67	1	Winter	2.5	mountain
5	1010.03	2	Summer	5.0	inland
6	990.87	1	Winter	4.0	inland
7	984.46	1	Winter	3.5	inland
8	999.44	0	Winter	1.0	mountain
9	1012.13	8	Winter	7.5	coastal

	Weather Type
0	Rainy
1	Cloudy
2	Sunny
3	Sunny
4	Rainy
5	Cloudy
6	Snowy
7	Snowy
8	Snowy
9	Sunny

Gambar 20. Hasil Dataset

Output ini menampilkan tabel *DataFrame* yang berisi dataset latihan 3 yang telah dibuat. Terlihat ada 10 baris data dengan berbagai kolom fitur seperti Temperature, Humidity, Wind Speed, dan seterusnya, serta kolom target Weather Type di bagian akhir.

3) Preprocessing

```
3. Preprocessing (Encoding + Scalling)

encoders = {}

for col in ['Cloud Cover', 'Season', 'Location', 'Weather Type']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le # simpan encoder agar bisa inverse transform

# Target (encoded)
y = df['Weather Type']

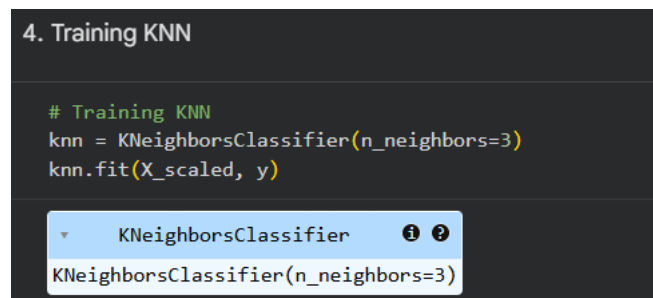
# Fitur (numerik + hasil encoding)
X = df.drop(columns=['Weather Type'])

# Scalling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Gambar 21. Preprocessing Encoding & Scalling

Pada tahap ini, saya melakukan persiapan data agar bisa diproses oleh algoritma KNN yang berbasis perhitungan jarak. Pertama, saya menggunakan LabelEncoder untuk mengubah semua data kategorikal (seperti Cloud Cover, Season, Location, dan target Weather Type) menjadi angka, di mana setiap *encoder* saya simpan dalam dictionary *encoders* agar nantinya hasil prediksi bisa dikembalikan ke teks aslinya (*inverse transform*). Setelah memisahkan fitur (X) dan target (y), saya wajib melakukan *Feature Scaling* menggunakan StandardScaler pada seluruh fitur X; langkah ini sangat krusial untuk menyamakan skala data (misalnya agar nilai Tekanan Udara yang ribuan tidak "mengalahkan" Kecepatan Angin yang satuan) sehingga perhitungan jarak Euclidean menjadi akurat dan model bisa bekerja optimal.

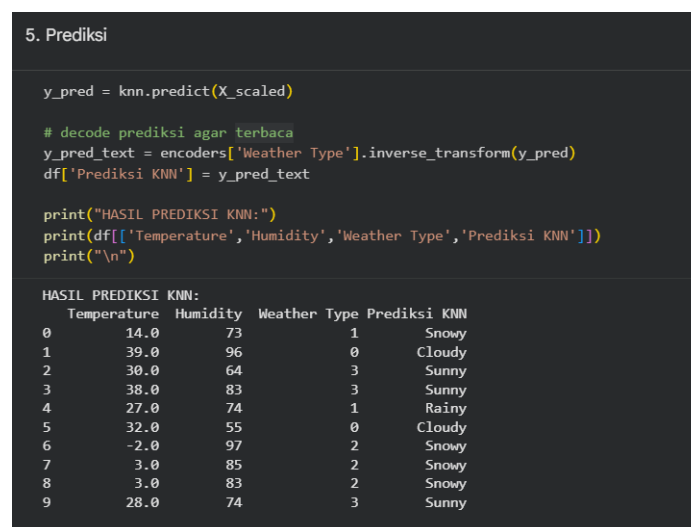
4) Training KNN



Gambar 22. Training KNN

Pada bagian ini, saya melatih model KNN menggunakan kelas KNeighborsClassifier dari *library* scikit-learn. Saya menginisialisasi model dengan parameter `n_neighbors=3`, yang berarti algoritma akan mempertimbangkan 3 tetangga terdekat untuk menentukan klasifikasi setiap data baru. Setelah model dibuat, saya melatihnya dengan memanggil fungsi `.fit()` menggunakan data fitur yang sudah disetarakan skalanya (`X_scaled`) dan data target (`y`); proses ini memungkinkan model mempelajari pola hubungan antara fitur cuaca dengan jenis cuacanya.

5) Prediksi

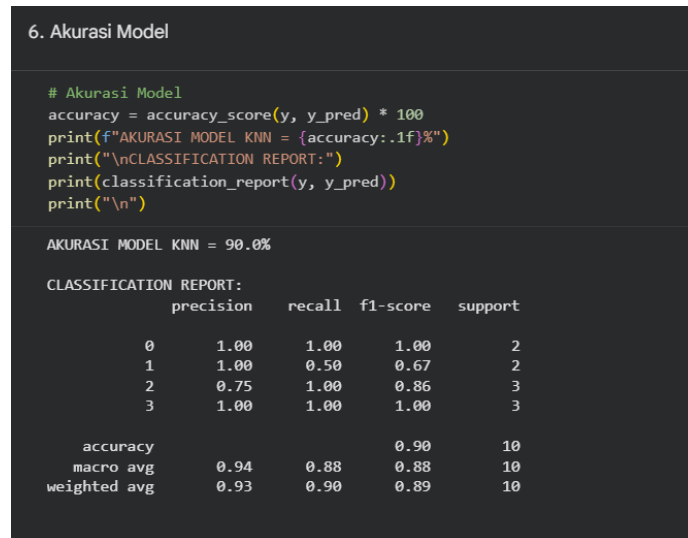


Gambar 23. Prediksi

Pada tahap ini, saya menggunakan model KNN yang telah dilatih untuk melakukan prediksi terhadap seluruh data menggunakan fitur yang sudah discaling (`X_scaled`). Karena

keluaran awal dari model berupa nilai numerik (hasil encoding), saya perlu mengembalikannya ke bentuk teks asli agar mudah dibaca. Saya menggunakan metode `.inverse_transform()` dari *encoder* target (Weather Type) yang sudah disimpan sebelumnya untuk mengubah angka kembali menjadi label cuaca (seperti Rainy, Cloudy, dll). Hasil prediksi teks tersebut kemudian saya tambahkan ke dalam dataframe sebagai kolom baru bernama 'Prediksi KNN' dan saya tampilkan sebagian datanya untuk membandingkan antara fitur input dan hasil prediksi model.

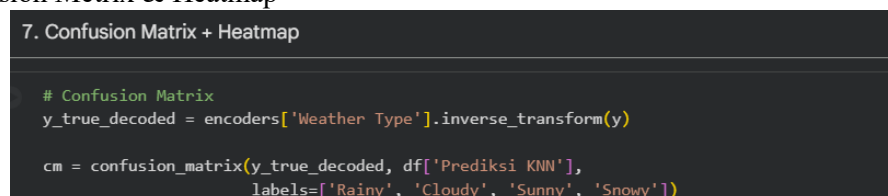
6) Akurasi Model



Gambar 24. Akurasi Model

Pada tahap ini, saya melakukan evaluasi kinerja model KNN dengan menghitung tingkat akurasinya. Fungsi `accuracy_score` dari `scikit-learn` digunakan untuk membandingkan label target asli (`y`) dengan hasil prediksi model (`y_pred`). Nilai akurasi kemudian dikalikan 100 agar tampil dalam format persentase. Hasil output menunjukkan akurasi model sebesar 90.0%, yang berarti model berhasil memprediksi dengan benar 9 dari 10 data yang diujikan. Selain itu, saya juga menampilkan `classification_report` untuk melihat detail metrik evaluasi lainnya seperti *precision*, *recall*, dan *f1-score* untuk setiap kelas cuaca, yang memberikan gambaran lebih lengkap tentang performa model pada masing-masing kategori.

7) Confusion Metrix & Heatmap



Gambar 25. Confusion Metrix

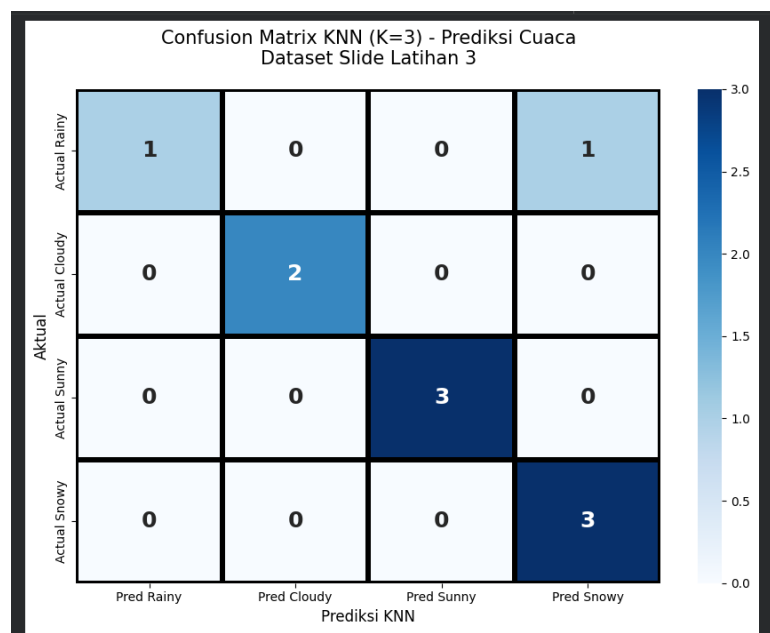
Pada bagian ini, saya membuat *Confusion Matrix* untuk memvisualisasikan performa model klasifikasi secara lebih detail. Saya membandingkan label target asli yang sudah di-*decode* (`y_true_decoded`) dengan hasil prediksi model (`df['Prediksi KNN']`). Matriks ini disusun berdasarkan label kelas cuaca ['Rainy', 'Cloudy', 'Sunny', 'Snowy']

untuk melihat berapa banyak prediksi yang benar (*True Positive*) dan kesalahan klasifikasi antar kelas (*Misclassification*).

```
# Heatmap
plt.figure(figsize=(9,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', linewidths=3, linecolor='black',
            xticklabels=['Pred Rainy', 'Pred Cloudy', 'Pred Sunny', 'Pred Snowy'],
            yticklabels=['Actual Rainy', 'Actual Cloudy', 'Actual Sunny', 'Actual Snowy'],
            annot_kws={"size": 18, "weight": "bold"})
plt.title('Confusion Matrix KNN (K=3) - Prediksi Cuaca\nDataset Slide Latihan 3',
          fontsize=15, pad=20)
plt.ylabel('Aktual', fontsize=12)
plt.xlabel('Prediksi KNN', fontsize=12)
plt.tight_layout()
plt.savefig('reports/latihan3_knn_cuaca_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()
```

Gambar 26. Heatmap

Pada bagian ini, saya menggunakan pustaka seaborn untuk mengubah confusion matrix menjadi *heatmap* yang informatif. Saya mengatur ukuran plot, warna (cmap='Blues'), dan menambahkan label sumbu yang jelas untuk memudahkan pembacaan hasil prediksi versus data aktual. Angka di dalam setiap kotak *heatmap* menunjukkan jumlah data yang diprediksi ke dalam kategori tertentu, sehingga saya dapat dengan cepat mengidentifikasi di mana model melakukan kesalahan atau sudah memprediksi dengan benar.



Gambar 27. Hasil Heatmap

Visualisasi *heatmap* dari *Confusion Matrix* di atas memberikan gambaran detail mengenai kinerja klasifikasi model KNN dengan K=3. Sumbu horizontal merepresentasikan hasil prediksi model, sedangkan sumbu vertikal menunjukkan label aktual. Dari grafik tersebut, terlihat jelas bahwa mayoritas data terkonsentrasi pada garis diagonal utama (kotak berwarna lebih gelap), yang menandakan prediksi yang tepat (*True Positive*). Secara spesifik, model berhasil memprediksi dengan benar 2 data *Cloudy*, 3 data *Sunny*, 3 data *Snowy*, dan 1 data *Rainy*. Kesalahan prediksi (*misclassification*) hanya terjadi pada satu sampel data, di mana kondisi cuaca yang seharusnya *Rainy* (Hujan) justru diprediksi sebagai *Snowy* (Salju). Dominasi warna pada diagonal utama dan minimnya nilai

di luar diagonal ini mengonfirmasi bahwa model memiliki performa yang sangat baik dengan tingkat akurasi mencapai 90%.

Kesimpulan Latihan 3:

Berdasarkan hasil Latihan 3, dapat disimpulkan bahwa penerapan algoritma K-Nearest Neighbors (KNN) dengan konfigurasi $K=3$ terbukti sangat efektif dalam memprediksi jenis cuaca, mencapai tingkat akurasi sebesar 90%. Keberhasilan model ini sangat bergantung pada tahapan *preprocessing* data yang tepat, khususnya penggunaan Feature Scaling (*StandardScaler*) yang krusial untuk menyeimbangkan skala nilai antar fitur agar perhitungan jarak Euclidean menjadi valid. Hal ini membuktikan bahwa dengan penanganan data yang benar, algoritma KNN mampu mengenali pola karakteristik cuaca yang kompleks dan memberikan hasil klasifikasi yang presisi dengan tingkat kesalahan yang sangat minim.

Link Github

Praktikum 10:

<https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum10/notebook/praktikum10.ipynb>

Praktikum Mandiri:

https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum10/notebook/praktikum_mandiri10.ipynb